

AST and Parser Deep Dive

Project: /home/adminn/BytecodeVM_Interpreter

Focus: Parser concepts, AST concepts, project implementation, examples, and dry runs

Date: 2026-05-17

1. Where Parser And AST Fit In The Interpreter

Your project executes a small programming language. The parser and AST sit in the middle of the language pipeline. They are the bridge between simple tokens and executable bytecode.

```
Source Code
  |
  v
Lexer
  |
  v
Tokens
  |
  v
Parser
  |
  v
AST
  |
  v
Compiler
  |
  v
Bytecode
  |
  v
Virtual Machine
```

The lexer only understands small pieces: numbers, names, operators, braces, and keywords. The parser understands structure. It answers questions like:

- Is this token sequence an assignment?

- Is this expression an addition or multiplication?
- Does this `if` have an `else`?
- Where does a function body start and end?
- Which operation should happen first?

The AST is the parser's output. It is a tree-shaped representation of the program's meaning. Instead of keeping the source as text, the project stores it as objects like `BinaryOpNode`, `IfNode`, `WhileNode`, and `FunctionDefNode`.

2. What Is A Parser?

A parser is a program that reads a sequence of tokens and checks whether they form a valid structure according to a grammar. If the structure is valid, the parser builds a useful representation of that structure. In this project, that representation is an AST.

Example source:

```
x = 2 + 3 * 4
```

The lexer produces tokens:

```
IDENTIFIER("x")
EQUAL("=")
NUMBER("2")
PLUS("+")
NUMBER("3")
STAR("*")
NUMBER("4")
EOF_TOKEN("")
```

The parser must understand that this is not just a flat list. It is an assignment where the right-hand side is an arithmetic expression. It must also understand that multiplication has higher precedence than addition.

```
AssignNode("x")
├─ BinaryOpNode("+")
│   └─ NumberNode("2")
└─ BinaryOpNode("*")
    ├── NumberNode("3")
    └─ NumberNode("4")
```

3. What Is An AST?

AST means **Abstract Syntax Tree**. It is a tree representation of source code. The word "syntax" means structure. The word "abstract" means it removes unnecessary surface details.

For example, the source has exact characters, spacing, and punctuation:

```
( 2 + 3 ) * 4
```

The AST does not care about the spaces. It also does not usually store parentheses as real nodes. Parentheses only influence the final tree shape:

```
BinaryOpNode("*")
├─ BinaryOpNode("+")
│   ├── NumberNode("2")
│   └─ NumberNode("3")
└─ NumberNode("4")
```

This tree says: first compute $2 + 3$, then multiply the result by 4.

Key idea: source code is written for humans, but an AST is structured for the compiler.

4. Concrete Syntax Tree vs Abstract Syntax Tree

A complete parse tree, sometimes called a concrete syntax tree, includes every grammar detail. It may include parentheses, punctuation, and intermediate grammar rules. An AST is cleaner. It keeps only the meaningful program structure.

Concrete Syntax Tree	Abstract Syntax Tree
Contains all syntax details.	Contains semantic structure.
Can include parentheses, commas, grammar helper nodes.	Usually removes punctuation and helper nodes.
Useful for exact syntax analysis.	Useful for compilation and execution.
Closer to source text.	Closer to program meaning.

5. AST Classes In Your Project

Your AST classes are declared in `ast.h`. All node classes inherit from the base class `AST`. The compiler later uses `dynamic_cast` to check which exact node type it is compiling.

5.1 Base AST Class

```
class AST {
public:
    virtual ~AST() {}
};
```

This base class makes all AST node types part of the same family. The virtual destructor also makes `dynamic_cast` work because the class becomes polymorphic.

5.2 NumberNode

```
class NumberNode : public AST {
public:
    string value;

    NumberNode(string val) {
        value = val;
    }
};
```

Represents integer literals such as 5, 100, or 42. It stores the number as a string because token values are strings. The VM later converts constants using `stoi`.

5.3 BinaryOpNode

```
class BinaryOpNode : public AST {
public:
    AST* left;
    string op;
    AST* right;
};
```

Represents arithmetic operations such as `2 + 3`, `x - 1`, or `n * fact(n - 1)`.

```

BinaryOpNode("")
├─ VariableNode("n")
└─ FunctionCallNode("fact")
    └─ BinaryOpNode("-")
        ├── VariableNode("n")
        └─ NumberNode("1")

```

5.4 VariableNode

```

class VariableNode : public AST {
public:
    string name;
};

```

Represents reading a variable. For example, in $x + 1$, x becomes a `VariableNode`. The compiler turns it into `OP_LOAD_NAME x`.

5.5 AssignNode

```

class AssignNode : public AST {
public:
    string name;
    AST* value;
};

```

Represents assigning to a variable. In $x = 2 + 3$, the node stores `name = "x"` and the value is a subtree for $2 + 3$.

5.6 ProgramNode

```

class ProgramNode : public AST {
public:
    vector<AST*> statements;
};

```

Represents the entire program. It contains all top-level statements in source order.

```

ProgramNode
├─ FunctionCallNode("fact")

```

```
└─ FunctionDefNode("fact")
```

5.7 IfNode

```
class IfNode : public AST {  
public:  
    AST* condition;  
    vector<AST*> ifBody;  
    vector<AST*> elseBody;  
};
```

Represents an `if` statement. The condition is an expression tree. The if body and else body are lists of statements.

5.8 CompareNode

```
class CompareNode : public AST {  
public:  
    AST* left;  
    string op;  
    AST* right;  
};
```

Represents comparisons such as `n == 0`, `i < 5`, or `x > y`. Comparisons produce integer boolean values in the VM: 1 for true and 0 for false.

5.9 WhileNode

```
class WhileNode : public AST {  
public:  
    AST* condition;  
    vector<AST*> body;  
};
```

Represents a while loop. The parser stores the loop condition separately from the body statements.

5.10 FunctionDefNode

```
class FunctionDefNode : public AST {
```

```
public:
    string name;
    vector<string> params;
    vector<AST*> body;
};
```

Represents a function definition such as:

```
func fact(n) {
    ...
}
```

It stores the function name, parameter names, and body statements.

5.11 FunctionCallNode

```
class FunctionCallNode : public AST {
public:
    string name;
    vector<AST*> args;
};
```

Represents a function call such as `fact(5)` or `add(x, y)`. Each argument is itself an AST expression.

6. Parser Type Used: Recursive Descent Parser

Your project uses a **recursive descent parser**. This means the grammar is implemented as a group of functions, where each function parses one kind of language construct.

For example:

- `program()` parses the whole program.
- `statement()` parses one statement.
- `expr()` parses addition and subtraction.
- `term()` parses multiplication and division.
- `factor()` parses numbers, variables, function calls, and parentheses.

The name "recursive descent" comes from the fact that functions call other parser functions to descend into smaller pieces of syntax. For example, `expr()` calls `term()`, and `term()` calls `factor()`.

7. Parser State In Your Project

```
class Parser {
public:
    vector<Token> tokens;
    int pos;
    Token currentToken;

    Parser(vector<Token> t);
    void advance();
    AST* parse();
    AST* expr();
    AST* term();
    AST* factor();
    AST* statement();
    AST* program();
    AST* ifStatement();
    AST* comparison();
    AST* whileStatement();
    AST* functionDefinition();
    AST* functionCall();
};
```

Field	Meaning
tokens	The full token list from the lexer.
pos	The current token index.
currentToken	The token currently being parsed.

7.1 Parser Constructor

```
Parser::Parser(vector<Token> t) {
    tokens = t;
    pos = 0;
    currentToken = tokens[pos];
}
```

The parser starts at token index zero. It assumes the token vector is not empty. This is safe in normal flow because the lexer always appends EOF_TOKEN.

7.2 advance()

```
void Parser::advance() {
    pos++;

    if (pos < tokens.size()) {
        currentToken = tokens[pos];
    }
}
```

`advance()` consumes the current token and moves to the next one. Parser code calls `advance()` after it has recognized a token.

8. Grammar Of Your Language

A grammar describes how valid programs are built from tokens. Your parser implements this approximate grammar:

```
program      -> statement* EOF

statement    -> functionDefinition
              | ifStatement
              | whileStatement
              | assignment
              | comparison

assignment   -> IDENTIFIER "=" comparison

comparison   -> expr ((">" | "<" | "==") expr)*

expr         -> term (("+ " | "- ") term)*

term         -> factor (("*" | "/" ) factor)*

factor       -> NUMBER
              | IDENTIFIER
              | functionCall
              | "(" comparison ")"

ifStatement  -> "if" comparison "{" statement* "}"
              ("else" "{" statement* "}")?

whileStatement -> "while" comparison "{" statement* "}"
```

```
functionDefinition
    -> "func" IDENTIFIER "(" params? ")" "{" statement* "}"

functionCall    -> IDENTIFIER "(" args? ")"
```

This grammar explains why the parser functions are structured as they are.

9. Operator Precedence Deep Dive

Operator precedence decides which operation binds more tightly. In normal math, multiplication happens before addition:

```
2 + 3 * 4 = 14
not 20
```

Your parser enforces this using separate functions:

Parser Function	Handles	Precedence
<code>factor()</code>	Numbers, variables, function calls, parentheses	Highest
<code>term()</code>	<code>*</code> , <code>/</code>	High
<code>expr()</code>	<code>+</code> , <code>-</code>	Medium
<code>comparison()</code>	<code>></code> , <code><</code> , <code>==</code>	Low

The lower-precedence function calls the higher-precedence function first. For example, `expr()` starts by parsing a `term()`. That means multiplication gets grouped before addition.

10. Deep Dive: `factor()`

`factor()` parses the smallest meaningful expression units.

It handles four cases:

- A number, such as 123.
- An identifier, such as x.
- A function call, such as fact(5).
- A parenthesized expression, such as (2 + 3).

Important function-call detection:

```
if (currentToken.type == IDENTIFIER) {
    if (tokens[pos + 1].type == LPAREN) {
        return functionCall();
    }

    AST* node = new VariableNode(currentToken.value);
    advance();
    return node;
}
```

The parser looks one token ahead. If an identifier is followed by (, it is treated as a function call. Otherwise, it is treated as a variable read.

Safety note: tokens[pos + 1] assumes there is another token available. The lexer appends EOF_TOKEN, so valid input usually works, but production parsers should still bounds-check.

11. Deep Dive: term()

term() handles multiplication and division. It starts by parsing a factor(), then keeps consuming * or / operators.

```
AST* Parser::term() {
    AST* node = factor();

    while (
        currentToken.type == STAR ||
        currentToken.type == SLASH
    ) {
        string op = currentToken.value;
        advance();
        AST* right = factor();
        node = new BinaryOpNode(node, op, right);
    }
}
```

```
    return node;
}
```

11.1 Example

```
3 * 4 / 2
```

The parser builds this left-associative tree:

```
BinaryOpNode("/")
├─ BinaryOpNode("*")
│   ├─ NumberNode("3")
│   └─ NumberNode("4")
└─ NumberNode("2")
```

That means the expression is interpreted as:

```
(3 * 4) / 2
```

12. Deep Dive: `expr()`

`expr()` handles addition and subtraction. It uses `term()` for operands, so multiplication and division are already grouped before addition and subtraction.

```
AST* Parser::expr() {
    AST* node = term();

    while (
        currentToken.type == PLUS ||
        currentToken.type == MINUS
    ) {
        string op = currentToken.value;
        advance();
        AST* right = term();
        node = new BinaryOpNode(node, op, right);
    }

    return node;
}
```

12.1 Example

```
2 + 3 * 4
```

`expr()` first gets 2 from `term()`. Then it sees `+`. For the right side, it calls `term()`, which consumes `3 * 4` as one subtree.

```
BinaryOpNode("+")
├─ NumberNode("2")
└─ BinaryOpNode("*")
    ├─ NumberNode("3")
    └─ NumberNode("4")
```

13. Deep Dive: `comparison()`

`comparison()` parses comparison operators: `>`, `<`, and `==`. It calls `expr()` for both left and right operands.

```
AST* Parser::comparison() {
    AST* node = expr();

    while (
        currentToken.type == GREATER ||
        currentToken.type == LESS ||
        currentToken.type == EQUAL_EQUAL
    ) {
        string op = currentToken.value;
        advance();
        AST* right = expr();
        node = new CompareNode(node, op, right);
    }

    return node;
}
```

13.1 Example

```
n == 0
```

AST:

```
CompareNode("==")
├─ VariableNode("n")
└─ NumberNode("0")
```

14. Deep Dive: statement()

`statement()` is the parser's dispatcher. It decides what kind of statement starts at the current token.

```
AST* Parser::statement() {
    if (currentToken.type == FUNC) {
        return functionDefinition();
    }

    if (currentToken.type == IF) {
        return ifStatement();
    }

    if (currentToken.type == WHILE) {
        return whileStatement();
    }

    if (
        currentToken.type == IDENTIFIER &&
        tokens[pos + 1].type == EQUAL
    ) {
        string name = currentToken.value;
        advance();
        advance();
        AST* value = comparison();
        return new AssignNode(name, value);
    }

    return comparison();
}
```

The order matters. If the current token is `FUNC`, the parser must not treat it as an expression. If the current token is an identifier followed by `=`, it is an assignment. Otherwise, the parser treats the statement as an expression.

15. Deep Dive: program()

`program()` parses the whole file or source string. It keeps reading statements until the lexer-provided `EOF_TOKEN`.

```
AST* Parser::program() {
    vector<AST*> statements;

    while (currentToken.type != EOF_TOKEN) {
        statements.push_back(statement());
    }

    return new ProgramNode(statements);
}
```

The result is a `ProgramNode` containing all top-level statements.

16. Deep Dive: ifStatement()

An `if` statement has a condition, an if-body, and optionally an else-body.

```
if x > 5 {
    x * 2
}
else {
    x - 2
}
```

AST shape:

```
IfNode
├─ condition:
│   └─ CompareNode(">")
│       ├── VariableNode("x")
│       └─ NumberNode("5")
├─ ifBody:
│   └─ BinaryOpNode("*")
│       ├── VariableNode("x")
│       └─ NumberNode("2")
└─ elseBody:
    └─ BinaryOpNode("-")
```

```
├─ VariableNode("x")
└─ NumberNode("2")
```

The parser consumes `if`, parses the condition using `comparison()`, enters the brace block, collects statements until `}`, then optionally parses the `else` block.

17. Deep Dive: `whileStatement()`

A while loop has a condition and a body.

```
while i < 5 {
    i = i + 1
}
```

AST shape:

```
WhileNode
├─ condition:
│   └─ CompareNode("<")
│       ├── VariableNode("i")
│       └─ NumberNode("5")
└─ body:
    └─ AssignNode("i")
        ├── BinaryOpNode("+")
        │   ├── VariableNode("i")
        │   └─ NumberNode("1")
```

18. Deep Dive: `functionDefinition()`

Function definitions start with `func`, followed by a name, parameter list, and body.

```
func fact(n) {
    if n == 0 {
        1
    }
    else {
        n * fact(n - 1)
    }
}
```


AST shape:

```
FunctionDefNode("fact")
├─ params:
│   └─ "n"
└─ body:
    └─ IfNode
        ├─ condition:
        │   └─ CompareNode("==")
        │       ├─ VariableNode("n")
        │       └─ NumberNode("0")
        ├─ ifBody:
        │   └─ NumberNode("1")
        └─ elseBody:
            └─ BinaryOpNode("*")
                ├─ VariableNode("n")
                └─ FunctionCallNode("fact")
                    └─ BinaryOpNode("-")
                        ├─ VariableNode("n")
                        └─ NumberNode("1")
```

19. Deep Dive: functionCall()

Function calls start with an identifier followed by parentheses.

```
fact(5)
```

AST:

```
FunctionCallNode("fact")
└─ args:
    └─ NumberNode("5")
```

Function call arguments are parsed using `comparison()`. This means arguments can be full expressions:

```
fact(n - 1)
```

AST:

```
FunctionCallNode("fact")
```

```
└─ args:
  └─ BinaryOpNode("-")
    └─ VariableNode("n")
      └─ NumberNode("1")
```

20. Full Parser Walkthrough: $x = 2 + 3 * 4$

20.1 Tokens

```
0: IDENTIFIER("x")
1: EQUAL("=")
2: NUMBER("2")
3: PLUS("+")
4: NUMBER("3")
5: STAR("*")
6: NUMBER("4")
7: EOF_TOKEN("")
```

20.2 Function Call Flow

```
parse()
└─ program()
  └─ statement()
    └─ sees IDENTIFIER followed by EQUAL
    └─ consumes "x"
    └─ consumes "="
    └─ comparison()
      └─ expr()
        └─ term()
          └─ factor()
            └─ NumberNode("2")
          └─ sees PLUS
          └─ term()
            └─ factor()
              └─ NumberNode("3")
            └─ sees STAR
            └─ factor()
              └─ NumberNode("4")
```

20.3 Final AST

```
ProgramNode
├─ AssignNode("x")
│   └─ BinaryOpNode("+")
│       ├── NumberNode("2")
│       └─ BinaryOpNode("*")
│           ├── NumberNode("3")
│           └─ NumberNode("4")
```

21. Full Parser Walkthrough: Factorial Program

21.1 Source

```
fact(5)

func fact(n) {
    if n == 0 {
        1
    }
    else {
        n * fact(n - 1)
    }
}
```

21.2 Top-Level AST

```
ProgramNode
├─ FunctionCallNode("fact")
│   └─ NumberNode("5")
└─ FunctionDefNode("fact")
    ├── params: ["n"]
    └─ body:
        └─ IfNode
```

21.3 Expanded AST

```
ProgramNode
├─ FunctionCallNode("fact")
│   └─ args:
```

```

|   └─ NumberNode("5")
└─ FunctionDefNode("fact")
    └─ params:
        └─ "n"
    └─ body:
        └─ IfNode
            └─ condition:
                └─ CompareNode("==")
                    └─ VariableNode("n")
                        └─ NumberNode("0")
            └─ ifBody:
                └─ NumberNode("1")
            └─ elseBody:
                └─ BinaryOpNode("*")
                    └─ VariableNode("n")
                        └─ FunctionCallNode("fact")
                            └─ args:
                                └─ BinaryOpNode("-")
                                    └─ VariableNode("n")
                                        └─ NumberNode("1")

```

21.4 Parser Concept

The parser does not execute recursion. It only recognizes that the function body contains a call to `fact`. Recursion happens later in the VM when bytecode execution reaches `OP_CALL fact`.

Important distinction: the parser builds the structure of recursive code. The VM performs recursive execution.

22. Common Beginner Confusions

22.1 Does The Parser Calculate $2 + 3$?

No. The parser does not evaluate expressions. It only builds a tree that says an addition should happen later.

```

BinaryOpNode("+")
└─ NumberNode("2")
└─ NumberNode("3")

```

The compiler turns that into bytecode, and the VM performs the actual addition.

22.2 Does The AST Store Every Token?

No. The AST does not store every token. It stores meaning. For example, braces are not AST nodes. Braces only tell the parser where a block starts and ends.

22.3 Why Are There Separate BinaryOpNode And CompareNode?

The project uses `BinaryOpNode` for arithmetic and `CompareNode` for comparisons. This is not the only possible design. Some interpreters use one generic binary operation node for both. This project separates them so the compiler can emit arithmetic opcodes separately from comparison opcodes.

22.4 Why Does Function Body Use A Vector?

A function body can contain multiple statements. Therefore `FunctionDefNode` stores:

```
vector<AST*> body;
```

The same pattern is used for program statements, `if` body, `else` body, and `while` body.

23. Parser Limitations In This Project

Limitation	Explanation
No syntax errors	The parser assumes valid input and does not produce helpful error messages.
No line/column tracking	Tokens do not store source locations, so the parser cannot report exact error positions.
Unsafe lookahead	Uses <code>tokens[pos + 1]</code> without explicit bounds checks.
No unary minus	<code>-5</code> is not parsed as one negative number or unary expression.
No explicit return statement	Function return behavior is based on the final value left by compiled body expressions.
Raw pointer ownership	AST nodes are allocated with <code>new</code> and are not deleted.

24. How A Production Parser Would Improve This

- Add token source locations: line and column.
- Add an `expect ( )` helper for required tokens.
- Report syntax errors instead of silently continuing.
- Use `unique_ptr<AST>` instead of raw pointers.
- Add unary expression support.
- Add a return statement node.
- Add better block parsing helpers to avoid duplicate brace logic.
- Add tests for every grammar rule.

24.1 Example expect Helper

```
Token Parser::expect(TokenType expected, string message) {
    if (currentToken.type != expected) {
        throw runtime_error(message);
    }

    Token token = currentToken;
    advance();
    return token;
}
```

Then parsing a parenthesized expression could say:

```
expect(LPAREN, "Expected '('");
AST* node = comparison();
expect(RPAREN, "Expected ')'");
```

25. How To Think Like A Compiler Engineer

When reading parser code, always ask:

- What grammar rule is this function implementing?
- What tokens does it consume?
- What AST node does it return?
- What happens if the input is invalid?
- How does this function preserve precedence?
- Is the parse left-associative or right-associative?

- Does the AST represent syntax or meaning?

For your project, the most important mental model is:

```
Tokens are flat.
Parser finds structure.
AST stores meaning.
Compiler lowers meaning to bytecode.
VM executes bytecode.
```

26. Practice Exercises

26.1 Draw The AST

For this program:

```
x = (1 + 2) * 3
```

Expected AST:

```
AssignNode("x")
├─ BinaryOpNode("*")
│   ├─ BinaryOpNode("+")
│   │   ├─ NumberNode("1")
│   │   └─ NumberNode("2")
│   └─ NumberNode("3")
```

26.2 Predict The Parser Functions

For this expression:

```
a + b * c
```

Function flow:

```
comparison()
├─ expr()
│   ├─ term()
│   │   └─ factor() -> VariableNode("a")
```

```

├─ PLUS
└─ term()
    ├─ factor() -> VariableNode("b")
    └─ STAR
        └─ factor() -> VariableNode("c")

```

26.3 Identify The Node Type

Source	Main AST Node
10	NumberNode
x	VariableNode
x = 10	AssignNode
x + 1	BinaryOpNode
x == 1	CompareNode
fact(5)	FunctionCallNode
func f(x) { x }	FunctionDefNode

27. Final Summary

The parser and AST are the core understanding layer of this interpreter. The lexer only gives a flat token stream. The parser turns that stream into a meaningful tree. The AST then becomes the input for compilation.

In this project:

- `parser.cpp` implements a recursive descent parser.
- `ast.h` defines the tree nodes.
- Operator precedence is handled by separating `factor`, `term`, `expr`, and `comparison`.
- Statements are selected by `statement()`.
- The final program tree is wrapped in `ProgramNode`.
- The compiler later walks the AST and emits bytecode.

Once you understand the parser and AST, the rest of the project becomes easier: the compiler simply maps

each AST node type to bytecode, and the VM executes that bytecode.